Generating a Phylogenetic Tree Using Pairwise MASH Distances and Neighbor Joining Algorithm

Aditya Kamat | Devansh Gupta

Overview of the problem

A core challenge in comparative genomic analysis is to determine genomic similarity and evolutionary relationships between organisms. While multiple sequence alignment (MSA) is commonly used to construct phylogenetic trees, such methods are computationally intensive and do not scale well to large datasets. The pairwise Mash distance problem addresses this by using MinHash sketching (dimensionality reduction) to estimate genomic similarity. By hashing a large genome sequence into a smaller *sketch*, the Jaccard index J between sketches is used to estimate the similarity between a pair of sequences.

Using J, the Mash distance between two sequences is computed as

$$D = -\frac{1}{k} \ln\left(\frac{2J}{1+J}\right),$$

where k is the k-mer size. The resulting $N \times N$ distance matrix can then be used to build a phylogenetic tree. Here N is the number of sequences to be compared.

While MSA has a computational complexity of $O(N^2L^2)$, Mash has complexity of $O(NL)$, making it far more scalable. Hence, Mash is an important method when working with large genome datasets, with applications such as whole-sequence comparisons, fast similarity screening, and large-scale genome clustering. It is also commonly used as a preprocessing step to select genomes for deeper analysis.

Current State-of-the-Art

An implementation of Mash is available at `marbl/Mash`. This repository will be used as a baseline to compare correctness and performance against our parallelized version. In addition to this baseline code, we shall add additional functionality to generate a phylogenetic tree from the computer MASH distances.

State-of-the-art methods related to Mash distance calculation include tools that use alternative sketching/hashing strategies rather than conventional MinHash (e.g., Dashing using HyperLogLog).

Another method, Skmer (SKetching genomes from MERs), adds a mathematical correction layer to account for sequencing error rate and genomic coverage; instead of using the plain Jaccard-based estimate directly, it uses a corrected distance formulation that boosts similarity scores based on estimated missing data.

How the Algorithm will be Parallelized

Platform: GPU (CUDA).

Algorithm steps:

1. **Generate Mash sketches (for all sequences):** map parallelism across sequences/blocks.
   (a) Create k-mers of a given sequence: map parallelism across threads within a block.
   (b) Hash the k-mers: map parallelism.
   (c) Sort hashed k-mers: parallel sort.
   (d) Retain the bottom n hashes (the sketch): no additional computation.
2. **Compute pairwise distances between sketches A_s and B_s:** Map parallelism across GPU Blocks (pair of sequences assigned to one block).
   (a) Compute $|A_s \cap B_s|$: Map parallelism across threads in a block.
   (b) Compute $|A_s \cup B_s|$ via $|A_s \cup B_s| = |A_s| + |B_s| - |A_s \cap B_s|$.
   (c) Compute Jaccard index $J = \frac{|A_s \cap B_s|}{|A_s \cup B_s|}$: Map parallelism across threads in a block.
   (d) Compute Mash distance D: Map parallelism across threads in a block.
   (e) Compute and store p-value Map parallelism across threads in a block.
3. **Build a tree using the Neighbor-Joining algorithm:**
   (a) Calculate the Q-matrix: map.
   (b) Find closest neighbors (minimum in Q): parallel min-reduction.
   (c) Update the distance matrix: map.

How Performance will be Measured and Compared

We will implement a sequential baseline by adapting the `mash` source code (`marbl/Mash/tree/master/src/mash`), since the repository does not directly provide an end-to-end pipeline for phylogenetic tree construction. We will then parallelize this baseline and report speedups using kernel execution time and host–device memory transfer time measurements.